Module 10 · SW Design

SW Design Project with AI

Everything from Phase 1, applied — build a real feature with Claude Code.

Akın Kaldıroğlu · Lead Trainer

The plan

Build Something Real

No new theory today. You'll take a small business problem from empty folder to tested, clean, well-designed code — with an AI agent as your pair.

Design first

Decide the shape before you generate code.

Build with AI

Direct Claude Code through the workflow you learned.

Hold the bar

Cohesion, coupling, SOLID, clean, secure, tested.

By the end of today

What You'll Produce

- A working solution to the business problem, on GitHub.
- A short design rationale — why this structure, these boundaries.
- Unit tests for the core logic, and honest error handling.
- Experience directing an agent instead of hand-typing everything.

The brief

The Business Problem

Small enough to finish; rich enough to design.

▶ runnable `dev.kaldiroglu.bootcamp.project`

A Library Loan Service

Build the core of a small library system. Keep it focused — the design is the point, not the feature count.

- Members can borrow and return books; each book has limited copies.
- A member may hold at most 5 loans at once; loans have a due date.
- Returning frees the copy — the next member can borrow it right away.
- Expose the operations behind a small, clean service API.

The brief

Your Design Checklist

Structure

- Cohesive types: Book, Copy, Member, Loan
- Loan rules isolated behind a boundary
- Depend on a Repository abstraction (DIP)

Quality

- Clean names & small functions
- Validated input; honest errors
- Unit tests for the loan rules

How to work

Direct the Agent

Apply the intent → plan → build → review loop.

How to work

Follow the Loop

1 · Brainstorm the design with the agent — agree it before coding

2 · Build one slice at a time — a rule, a test, a repository

3 · Review each slice with your design vocabulary

4 · Refactor with tests green; commit small and often

Small, reviewable steps beat one giant prompt — every time.

How to work

Prompting Tips for Today

- Give the agent the brief and your standards up front.
- Ask for the design and boundaries before any implementation.
- Name the patterns you want: 'loan rules as a Strategy', 'Repository port'.
- Make it write tests with the code — and run them each loop.

Logistics

How We Run It

Setup

- Any group size — solo or a team
- One shared GitHub repository per group
- Commit small, commit often

Timebox

- Design & agree the shape — first hour
- Build in slices — the middle
- Polish, test & write your rationale — last

Logistics

What 'Good' Looks Like

Design

Clear boundaries, high cohesion, low coupling, SOLID applied.

Quality

Clean, tested, secure — and it actually runs.

Process

Evidence you directed the AI: small commits, a clear rationale.

Watch out

Common Pitfalls

- Letting the agent generate everything before you've agreed a design.
- Accepting code you can't explain — if you can't review it, don't keep it.
- Skipping tests 'to save time' — they're what makes AI code trustworthy.
- Gold-plating: adding patterns the problem doesn't need.

Run it yourself

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 5 tests here

`project/LoanService`
the borrow / return rules

`project/Loan · Book`
the model the rules act on

`project/LoanRepository`
a port, with an in-memory adapter

C# / .NET

xUnit · 5 tests here

`Project.cs`
the Library Loan service

`ProjectTests.cs`
limit · availability · unknown book · return

Python

pytest · 5 tests here

`project.py`
the Library Loan service

`tests/test_project.py`
the same four rules

`github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · -DOTNET · -PYTHON`

This is today's core, not the graded project. The project extends it — and its loan limit is a constant, its due date a parameter. Both have to go.

Clone one, run the suite, then break something and watch which test goes red. ***Across all chapters the suites are 62 · 55 · 56 tests.***

Your project

▶ runnable `dev.kaldiroglu.bootcamp.project`

What You're Already Given

Today's core ships in all three companion repos. Clone it, run its five tests, and read those tests first — they say what the code actually promises.

In the box

- Book · Loan — the model
- LoanService — borrow, return
- LoanRepository — a port + adapter
- Named exceptions · 5 green tests

Not in the box

- Member, tiers, reservations
- Fines — no notion of them
- A notifier — nothing tells anyone
- Any policy you can swap

Your project

What You Must Build

Rule	The decision it forces	A surface to own
R1 · Borrow	The due date comes from the member's tier	Loans
R2 · Loan limit	The tier's maximum, not a constant	Loans
R3 · No copies → reserve	A failed borrow becomes a queue entry	Reservations
R4 · Return & hand-off	Free, price, notify, hold — one call, many jobs	Reservations
R5 · Overdue fine	A policy you can swap without touching return	Fines
R6 · Cancel reservation	The queue closes up behind the leaver	Reservations

Your project

What You Must Build, continued

Rule	The decision it forces	A surface to own
R7 · Renew a loan	Refused if anyone is waiting; the tier caps it	Loans
R8 · Holds expire	'Now' becomes explicit — you can't trust today	Reservations
R9 · Fines block borrowing	A second policy, consulted by borrowing	Fines
R10 · Catalogue admin	Withdrawing a copy on loan must fail	Catalogue
R11 · Activity log	Every action through a port you own	Cross-cutting

Eleven rules, five surfaces. A surface is something one person can own end to end — that is how a team splits this without five people editing one class.

Your project

Out of Scope — Deliberately

The domain is the whole deliverable. Everything outside it is a port you own, with an in-memory implementation behind it.

Not in this project

- No UI — none, of any kind
- No controller, no REST, no HTTP
- No database, no ORM, no SQL
- No framework — plain classes

Your only outside edges

- Repository — where loans live
- Notifier — telling a reserver
- Activity log — what happened
- Clock — how you know 'today'

A main() that calls the service is demo enough — or just the test suite.

Your project

Extend It, or Start Clean?

Start from the lecture code or from an empty folder — both are graded the same. Either way, four things in today's core cannot survive the brief:

- **The loan limit is a constant — it belongs to the tier, not to LoanService**
- **The due date is a parameter — the domain must derive it, so where does 'today' come from?**
- **returnBook() is one line — it becomes fine, queue, hold and notify. What stays out?**
- **Availability is two counts — a held copy is a third state the model can't express**

Growing the lecture class without revisiting the model is the failure mode we look for first.

Your project

Build It — and Get Graded

Today you build the core; the graded project extends it with membership tiers, swappable fines, and reservations. Read the rubric before you start.

Project 10 — Library Loan Service (extended)

Brief `Project-10-Evaluation/PROJECT-BRIEF.md`

Rubric `Project-10-Evaluation/RUBRIC.md`

IN THIS SESSION, LOOK AT

- graded on design first (see the rubric bands)
- extra practice: the ACME Orders refactoring capstone

Remember these

Key Takeaways

- ✓ Design the shape before you generate the code.
- ✓ Build in small, reviewable slices — with tests each time.
- ✓ Direct the agent; review every slice with your design vocabulary.
- ✓ Commit small and often; keep a short design rationale.
- ✓ A clean, tested, well-bounded solution beats a big pile of features.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Domain-Driven Design** Eric Evans · Addison-Wesley, 2003

The blue book: ubiquitous language and model-driven design — the project's backbone.

📖 **Learning Domain-Driven Design** Vlad Khononov · O'Reilly, 2021

The fastest modern route into DDD — start here, then go back to Evans.

📖 **Refactoring, 2nd ed.** Martin Fowler · Addison-Wesley, 2018

The catalog of moves you will apply to whatever the AI hands you.

📖 **Implementing Domain-Driven Design** Vaughn Vernon · Addison-Wesley, 2013

The practical bridge from model to code — aggregates, repositories, services.

Next session

What's Next

Module 11 · Tuesday 4 August

Fundamentals of Software Architecture

with Akın Kaldıroğlu

Phase 2 begins: we zoom out from classes to whole systems — quality attributes, architecture models, and styles.

Let's Build.

Design first. Direct the agent. Hold the bar.

Akın Kaldıroğlu · SW Design Project with AI